

README / Setup Guide

Overview

This application is an employee leave management system designed to streamline the process of submitting, approving, and tracking leave requests within an organization. It provides a user-friendly interface for employees to submit leave requests and view their leave balances. Managers can approve or reject leave requests, and the system ensures that all data is securely managed and accessible only to authorized users.

Key Features

- **Employee Leave Request Submission:** Employees can submit leave requests with details such as leave type, start date, and end date.
- **Leave Request Approval Workflow:** Managers can approve or reject leave requests, updating the status accordingly.
- **Leave Balance Retrieval:** Employees can view their current leave balance and past leave requests.

Prerequisites

- **Java 11+**
- **PostgreSQL 13+**
- **Node.js 16+**
- **npm 8+**
- **Basic understanding of Spring Boot and React**

Project Structure

The repository is structured as follows:

- **backend/:** Contains the Spring Boot application with REST controllers, service classes, repository interfaces, and entity classes.
- **frontend/:** Contains the React application with components for submitting and viewing leave requests.
- **shared/:** Contains shared utilities and configurations.
- **docs/:** Contains API documentation and user guides.

Installation & Setup

Backend Setup

1. Clone the repository: `git clone <url>`
2. Navigate to the project: `cd employee-leave-management`
3. Create virtual environment: `mvn clean install`
4. Set environment variables: `export DB_URL=jdbc:postgresql://localhost:5432/leavedb` and `export DB_USER=leaveuser` and `export DB_PASS=leavepass`
5. Start server: `mvn spring-boot:run`
6. Verify: Backend is running on `http://localhost:8080`

Frontend Setup

1. Navigate to frontend: `cd frontend`
2. Install dependencies: `npm install`
3. Build/prepare assets: `npm run build`
4. Start dev server: `npm start`
5. Open browser: `http://localhost:3000`
6. Verify: UI loads and connects to backend

Configuration

Required Environment Variables

- `DB_URL`: Controls the PostgreSQL database URL, format: `jdbc:postgresql://<host>:<port>/<database>`, example value: `jdbc:postgresql://localhost:5432/leavedb`
- `DB_USER`: Controls the PostgreSQL database username, format: `string`, example value: `leaveuser`
- `DB_PASS`: Controls the PostgreSQL database password, format: `string`, example value: `leavepass`

Optional Environment Variables

- `JWT_SECRET` (default: `mysecret`): Controls the JWT secret key for authentication, effect if changed: new JWT tokens will be generated with the new secret key

Running the Application

Starting the Backend

1. Ensure virtual environment is activated

2. Set environment variables: `export DB_URL=jdbc:postgresql://localhost:5432/leavedb, export DB_USER=leaveuser, export DB_PASS=leavepass`
3. Start server: `mvn spring-boot:run`
4. Verify: Backend is running on `http://localhost:8080`

Starting the Frontend

1. Navigate to frontend directory: `cd frontend`
2. Start dev server: `npm start`
3. Open browser: `http://localhost:3000`
4. Verify: UI loads and connects to backend

Verification Checklist

- ☐ Backend health check responds at `GET /api/health` with 200 OK
- ☐ Frontend loads without console errors
- ☐ Basic workflow completes end-to-end (e.g., employee submits and manager approves a leave request)
- ☐ API endpoints respond with expected data format

Troubleshooting

Backend Issues

- **Port already in use:** Change `PORT` env var or kill process on port 8080
- **Module not found:** Run `mvn clean install` again
- **Database connection error:** Verify connection string in config, check service is running
- **Authentication error:** Verify JWT secret key in `.env` is correct and not expired

Frontend Issues

- **Blank page or won't load:** Check browser console for errors, ensure backend is running
- **API connection error:** Verify backend URL in config, check CORS settings
- **Module resolution error:** Delete `node_modules` and run `npm install` again
- **Build fails:** Ensure Node.js version matches requirements, clear cache: `npm cache clean --force`

Common Workflows

Submit Leave Request

1. Navigate to the leave request form
2. Fill in the leave request details (employee ID, leave type, start date, end date)
3. Click 'Submit'
4. Verify: Leave request is submitted and status is 'PENDING'

Approve Leave Request

1. Navigate to the leave request list
2. Click on the leave request to approve
3. Click 'Approve'
4. Verify: Leave request status is updated to 'APPROVED'

Deployment Notes

- Ensure PostgreSQL database is accessible from the deployed environment
- Set environment variables for database connection and JWT secret key
- Use HTTPS for secure communication

Getting Help

- Check logs: Backend logs in console, frontend logs in browser DevTools
- Review error message and stack trace for clues
- Check environment variables are set correctly
- Refer to the API Documentation and User Guide for more details